INFORME ACTIVIDAD CLASE 9

Construcción del perceptrón completo con descenso del gradiente

Campo	Detalle
Asignatura	Inteligencia Artificial - Ingeniería de Sistemas
Docente	Jhon James Cano Sánchez
Período	2026-02
Grupo	Número por confirmar
Integrante	Tatiana Vanessa Alvis
Modalidad	Desarrollo individual
Fecha de entrega	27-10-2026

Este informe documenta el diagnóstico de la neurona anterior, los siete retos y los resultados con 120 lotes simulados. El modelo estandariza precipitación y temperatura antes de entrenar; con tasa 0.1 alcanza 90 % de exactitud y pérdida 0.2281.

Tatiana Vanessa Alvis realizó la programación, verificación y relatoría en modalidad individual.

1. Diagnóstico Misión 0

La neurona de la Clase 8 se aplica a 120 registros simulados: 65 cosechas exitosas y 55 no exitosas. Su función escalón entrega una clase binaria, pero no una probabilidad.

¿Llega a cero errores?

No. Los errores fluctúan durante las 30 épocas: inician en 50.0, aumentan a 52.0 en la época 2 y finalizan en 31.0. No alcanzan cero ni disminuyen de manera sostenida.

¿Qué ocurre con el tamaño de los pesos?

Los pesos crecen en magnitud. Al inicio w1=w2=0.1; al cierre de la época 30, w1=33.86 y w2=-139.20. Las entradas están en escalas diferentes y la regla secuencial aplica actualizaciones grandes mientras recorre los datos.

¿Qué informa para el lote de 150 mm y 26 °C?

La neurona responde 1, pero la función escalón no expresa su nivel de seguridad. Esa salida sola no permite priorizar la asistencia técnica según una probabilidad.

¿Qué pasaría con un millón de lotes?

El ciclo secuencial procesaría cada lote de uno en uno. NumPy permite calcular las sumas ponderadas y gradientes de todas las filas mediante operaciones vectorizadas.

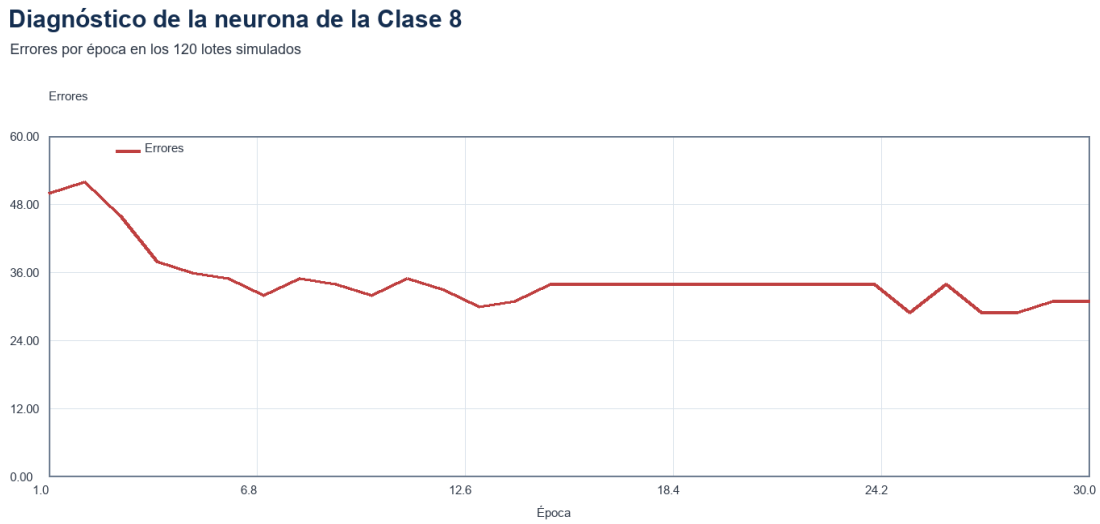


Figura 1. Errores por época de la neurona de la Clase 8.

Hipótesis inicial del grupo

Pendiente de completar con la hipótesis escrita antes de abrir el esqueleto. No se reconstruye retrospectivamente como si fuera un registro inicial.

2. Bitácora de retos

Se revisaron los siete puntos de control del esqueleto con el conjunto generado por la semilla 42. Los siete aparecen como [OK].

Reto 1 - Vectorización

Faltaba calcular z para todas las filas sin ciclos. El producto matriz-vector devuelve una suma ponderada por lote.

```
  z = X @ w + b
```

```
[OK] Reto 1 - Vectorización superado
```

Reto 2 - Sigmoide

La función escalón no da confianza. La sigmoide convierte z en probabilidad; se limita z para evitar desbordamiento.

```
  z = np.clip(z, -500, 500)
  p = 1 / (1 + np.exp(-z))
```

```
[OK] Reto 2 - Sigmoide superado
```

Reto 3 - Pérdida

El conteo de errores no mide cuán equivocada es una probabilidad segura. Se usa entropía cruzada binaria.

```
  -np.mean(y*np.log(p) + (1-y)*np.log(1-p))
```

```
[OK] Reto 3 - Pérdida superado
```

Reto 4A - Gradientes

Se calcula el cambio de pérdida respecto a cada peso y al bias, promediando todos los lotes.

```
  e = y_prob - y
  gw = X.T @ e / len(y)
  gb = np.mean(e)
```

```
[OK] Reto 4A - Gradientes superado
```

Reto 4B - Actualización

El descenso mueve pesos y bias en dirección opuesta al gradiente.

```
  w = w - tasa_aprendizaje * gradiente_w
  b = b - tasa_aprendizaje * gradiente_b
```

```
[OK] Reto 4B - Actualización superado
```

Reto 5 - Preparación de entradas

La lluvia y la temperatura tienen escalas distintas. Estandarizar centra las entradas y las deja con desviación estándar 1.

```
  media = X.mean(axis=0)
  desviacion = X.std(axis=0)
  X_esc = (X - media) / desviacion
```

```
[OK] Reto 5 - Preparación de entradas superado
```

Reto 6 - Frontera

La frontera corresponde a z=0, donde la probabilidad es 0.5; se despeja la segunda característica.

```
  x2_esc = -(w[0]*x1_esc + b) / w[1]
```

[OK] Reto 6 - Frontera superado

Reto 7 - Lotes nuevos

Los lotes nuevos se transforman con la media y desviación del conjunto de entrenamiento.

```
  lotes_nuevos_esc = (lotes_nuevos - media) / desviacion
```

[OK] Reto 7 - Lotes nuevos superado

3. Resultados

El dataset tiene 120 filas, 65 exitosas y 55 no exitosas. Media de entradas: lluvia 149.7183 mm y temperatura 25.8725 °C; desviaciones estándar 59.9735 mm y 3.0038 °C.

Medida	Sin estandarizar	Estandarizado
Pérdida inicial	0.6931	0.6931
Exactitud inicial	54.17 %	54.17 %
Pérdida final (500 épocas)	5.2830	0.2281
Exactitud final (500 épocas)	74.17%	90.00%
w1 / w2 / b	3.3155 / -25.2997 / 0.1440	2.6217 / -2.3658 / 0.3152
Advertencias	Desbordamiento en exp	Sin advertencias con z limitado

La comparación sin estandarizar usa 500 épocas y tasa 0.1. La sigmoide sin limitar su entrada genera desbordamientos para valores extremos. La solución limita z entre -500 y 500 antes de calcular exp.

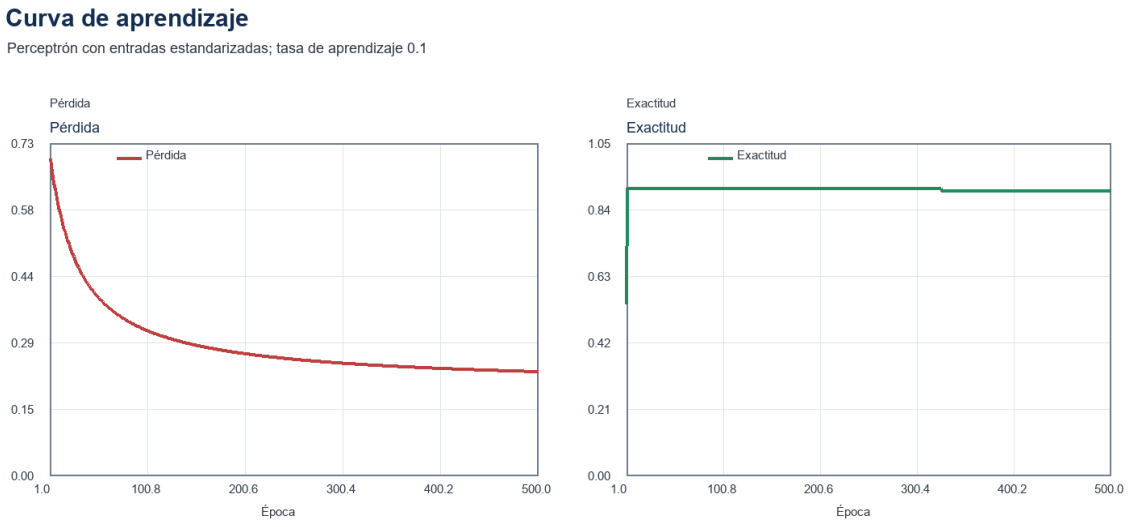


Figura 2. Pérdida y exactitud durante el entrenamiento estandarizado.

Interpretación de parámetros

w1=2.6217 es positivo: más precipitación aumenta z y la probabilidad estimada. w2=-2.3658 es negativo: más temperatura disminuye z. Son relaciones aprendidas de los datos simulados, no recomendaciones agronómicas reales.

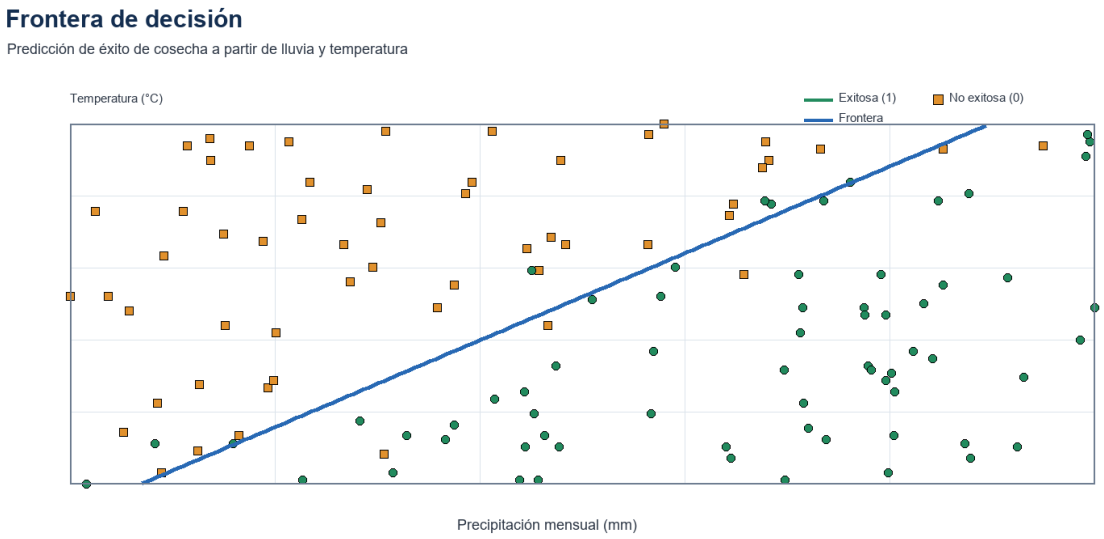


Figura 3. Frontera donde la probabilidad estimada es 0.5.

Lote nuevo	Probabilidad	Predicción
220 mm / 23.5 °C	99.48%	Exitosa
60 mm / 30 °C	0.11%	No exitosa
150 mm / 26 °C	55.65%	Exitosa, cerca de 0.5

4. Experimento

Se eligió el experimento A. Se varió la tasa de aprendizaje y se entrenaron 500 épocas con los datos estandarizados.

Tasa	Pérdida final	Exactitud
0.001	0.6289	90.83%
0.1 (base)	0.2281	90.00%
1.0	0.2050	90.00%

Con 0.001 el aprendizaje es más lento: pérdida 0.6289 y exactitud 90.83 %. Con 1.0 la pérdida es 0.2050 y la exactitud 90 %. En estos datos estandarizados, la tasa 1.0 reduce más la pérdida en 500 épocas; debe validarse con datos no usados en entrenamiento.

5. Hipótesis frente a realidad

La hipótesis previa no fue proporcionada. Completar con el registro original y comparar si se anticipó la necesidad de vectorizar, escalar, calcular probabilidades y medir la pérdida.

Hipótesis inicial escrita	¿Se confirmó? Evidencia
[Completar con el registro original]	[Comparar con retos y resultados]
[Completar]	[Completar]
[Completar]	[Completar]

6. Conclusiones

1. La neurona con función escalón no expresa incertidumbre y la regla secuencial no logra cero errores con este conjunto.

2. La vectorización y los gradientes permiten ajustar todos los ejemplos de cada época; la entropía cruzada mide el error de probabilidades.

3. La estandarización mejora la optimización. Las probabilidades describen datos simulados y deben validarse antes de aplicarse a cosechas reales.

Pregunta abierta

¿Cómo cambiarían la frontera y las probabilidades al entrenar con datos reales de varios ciclos agrícolas y evaluar en lotes no vistos?

Modalidad y roles

Tatiana Vanessa Alvis realizó programación, verificación y relatoría en modalidad individual. Número de grupo: pendiente. Fecha de entrega: 27-10-2026.

Anexo A. Salida final de consola

```
Datos: X tiene forma (120, 2) | y tiene forma (120,)
[OK] RETO 1 (vectorización) superado
[OK] RETO 2 (función sigmoide) superado
[OK] RETO 3 (función de pérdida) superado
[OK] RETO 4A (gradientes) superado
[OK] RETO 4B (actualización de pesos) superado
[OK] RETO 5 (preparación de las entradas) superado
[OK] RETO 6 (frontera de decisión) superado
[OK] RETO 7 (predicción de lotes nuevos) superado
Época 500 | Pérdida: 0.2281 | Exactitud: 90.00% | w=[2.622, -2.366] b=0.315
Pérdida final 0.2281 | Exactitud 90.00%
w1=2.6217, w2=-2.3658, b=0.3152
220 mm / 23.5 °C -> 99.48% -> EXITOSA
60 mm / 30 °C -> 0.11% -> NO EXITOSA
150 mm / 26 °C -> 55.65% -> EXITOSA
*** ¡Todos los retos superados! ***
```

Anexo B. Código completo resuelto

El archivo editable perceptron_reto.py se entrega por separado. A continuación se reproduce el código completo.

```
"""
perceptron_reto.py  –  Actividad grupal Clase 9
Construcción completa del perceptrón con descenso del gradiente.

¿CÓMO FUNCIONA ESTE ARCHIVO?
  - El código tiene 7 RETOS marcados con  >>> RETO X <<<.
  - Cada reto tiene un PUNTO DE CONTROL que verifica si quedó
    bien resuelto.
  - Al ejecutar el script, se detiene en el primer reto
    pendiente y muestra una pista.
  - Resuelva el reto, guarde (Ctrl + S) y vuelva a ejecutar:
    python perceptron_reto.py
  - Cuando todos los retos estén superados, el script genera los
    gráficos finales.

Genera: curva_aprendizaje.png, frontera_decision.png
"""


import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


def punto_de_control(superado, reto, pista):
    """Verifica un reto. Si no está superado, muestra la pista y
    detiene el programa."""
```

```
    if superado:
        print(f"[OK] {reto} superado")
    else:
        print(f"\n[PENDIENTE] {reto}")
        print(f"   Pista: {pista}")
        print("   Complete el reto, guarde con Ctrl + S y vuelva
    a ejecutar el script.")
        sys.exit()


# ============================================================
# 1. Cargar los datos
# ============================================================
df = pd.read_csv("../data/cultivos_cartago.csv")
X = df[["precipitacion_mm", "temperatura_c"]].to_numpy()
y = df["cosecha_exitosa"].to_numpy()
print(f"Datos: X tiene forma {X.shape} | y tiene forma
    {y.shape}\n")


# ============================================================
# >>> RETO 1 <<<  Vectorización: todas las sumas ponderadas a la
    vez
# ============================================================
def suma_ponderada(X, w, b):
    """Debe devolver z = w1*x1 + w2*x2 + b para TODAS las filas
    de X, sin usar ciclos."""
    return X @ w + b


X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
z_and = suma_ponderada(X_and, np.array([0.5, 0.5]), -0.7)
punto_de_control(
    z_and is not None and np.allclose(z_and, [-0.7, -0.2, -0.2,
    0.3]),
    "RETO 1 (vectorización)",
    "Con la neurona AND de la Clase 8 (w=[0.5, 0.5], b=-0.7) se
    esperaba [-0.7 -0.2 -0.2 0.3]. "
    "¿Qué operador de NumPy multiplica una matriz por un vector?
    (Clase 4)",
)


# ============================================================
# >>> RETO 2 <<<  Una activación que entregue probabilidades
# ============================================================
def sigmoide(z):
    """Debe convertir cualquier número z en un valor entre 0 y 1
    (0.5 cuando z = 0)."""
    z = np.clip(z, -500, 500)
```

```
    return 1 / (1 + np.exp(-z))


prueba = sigmoide(np.array([-6.0, 0.0, 6.0]))
punto_de_control(
    prueba is not None and np.allclose(prueba, [0.0025, 0.5,
    0.9975], atol=1e-3),
    "RETO 2 (función sigmoide)",
    "Para z = -6, 0 y 6 se esperaba aproximadamente 0.0025, 0.5
    y 0.9975. "
    "Busque la fórmula de la función sigmoide; en NumPy, e
    elevado a la x es np.exp(x).",
)


def predecir_probabilidad(X, w, b):
    return sigmoide(suma_ponderada(X, w, b))


# ============================================================
# >>> RETO 3 <<<  Medir qué tan mal predice el modelo
# ============================================================
def calcular_perdida(y_real, y_prob):
    """Pérdida logarítmica (entropía cruzada binaria). Debe
    devolver UN solo número."""
    epsilon = 1e-9
    y_prob = np.clip(y_prob, epsilon, 1 - epsilon)   # evita
    log(0)
    return -np.mean(y_real * np.log(y_prob) + (1 - y_real) *
    np.log(1 - y_prob))


perdida_inicial = calcular_perdida(np.array([1, 0, 1, 0]),
    np.array([0.5, 0.5, 0.5, 0.5]))
perdida_segura = calcular_perdida(np.array([1]),
    np.array([0.9]))
punto_de_control(
    perdida_inicial is not None and np.isclose(perdida_inicial,
    0.6931, atol=1e-3)
    and np.isclose(perdida_segura, 0.1054, atol=1e-3),
    "RETO 3 (función de pérdida)",
    "Si el modelo dice 0.5 para todo, la pérdida debe ser
    0.6931; si acierta con 0.9, debe ser 0.1054. "
    "Busque 'binary cross entropy'. Use np.log y np.mean.",
)


def calcular_exactitud(y_real, y_prob):
    return np.mean((y_prob >= 0.5).astype(int) == y_real)
# ============================================================
```

```
# >>> RETO 4 (parte A) <<<  ¿Hacia dónde mover los pesos?
# ============================================================
def calcular_gradientes(X, y_real, y_prob):
    """Debe devolver (gradiente_w, gradiente_b) promediando
    sobre todos los ejemplos."""
    n = len(y_real)
    diferencia = y_prob - y_real
    gradiente_w = X.T @ diferencia / n
    gradiente_b = np.mean(diferencia)
    return gradiente_w, gradiente_b


gw, gb = calcular_gradientes(np.array([[1.0, 0.0], [0.0, 1.0]]),
    np.array([1, 0]), np.array([0.5, 0.5]))
punto_de_control(
    gw is not None and np.allclose(gw, [-0.25, 0.25]) and
    np.isclose(gb, 0.0),
    "RETO 4A (gradientes)",
    "Con X=[[1,0],[0,1]], y=[1,0] y y_prob=[0.5,0.5] se esperaba
    gradiente_w=[-0.25 0.25] y gradiente_b=0. "
    "El error es (y_prob - y_real); el gradiente de w usa la
    transpuesta de X y se divide por n.",
)


# ============================================================
# >>> RETO 5 <<<  Preparar las entradas (resuélvalo cuando el
    punto de control lo pida)
# ============================================================
# Se calculan solo con los datos de entrenamiento y se
    reutilizan después.
media = X.mean(axis=0)
desviacion = X.std(axis=0)
if np.any(desviacion == 0):
    raise ValueError("Una característica no varía; no se puede
    estandarizar con desviación cero.")
X_esc = (X - media) / desviacion


# ============================================================
# 2. Entrenamiento con descenso del gradiente
# ============================================================
w = np.zeros(2)
b = 0.0
tasa_aprendizaje = 0.1
epocas = 500
historial_perdida = []
historial_exactitud = []

for epoca in range(1, epocas + 1):
```

```
    y_prob = predecir_probabilidad(X_esc, w, b)
    gradiente_w, gradiente_b = calcular_gradientes(X_esc, y,
    y_prob)

    # >>> RETO 4 (parte B) <<<  Dar el paso de aprendizaje
    # ESCRIBA AQUÍ SU SOLUCIÓN (complete las dos líneas de
    abajo)
    w = w - tasa_aprendizaje * gradiente_w
    b = b - tasa_aprendizaje * gradiente_b

    if epoca == 1:
        punto_de_control(
            np.allclose(w, -tasa_aprendizaje * gradiente_w) and
    np.isclose(b, -tasa_aprendizaje * gradiente_b),
            "RETO 4B (actualización de pesos)",
            "Los pesos no cambiaron o se movieron en la
    dirección equivocada. Recuerde la Clase 8: el nuevo peso "
            "depende del peso anterior, de la tasa de
    aprendizaje y del gradiente. ¿Se suma o se resta para ir
    'cuesta abajo'?",
        )

    historial_perdida.append(calcular_perdida(y, y_prob))
    historial_exactitud.append(calcular_exactitud(y, y_prob))
    if epoca == 1 or epoca % 100 == 0:
        print(f"Época {epoca:>3} | Pérdida:
    {historial_perdida[-1]:.4f} | "
              f"Exactitud: {historial_exactitud[-1]:.2%} |
    w=[{w[0]:.3f}, {w[1]:.3f}] b={b:.3f}")


y_prob_final = predecir_probabilidad(X_esc, w, b)
exactitud_final = calcular_exactitud(y, y_prob_final)
print(f"\nRESULTADO: pérdida final {calcular_perdida(y,
    y_prob_final):.4f} | exactitud {exactitud_final:.2%}")
print(f"Pesos finales: w1={w[0]:.4f}, w2={w[1]:.4f},
    b={b:.4f}\n")

punto_de_control(
    media is not None and desviacion is not None
    and np.allclose(X_esc.mean(axis=0), 0, atol=1e-6) and
    np.allclose(X_esc.std(axis=0), 1, atol=1e-6),
    "RETO 5 (preparación de las entradas)",
    "El modelo entrena, pero ¿los resultados son buenos?
    ¿Apareció alguna advertencia (RuntimeWarning)? "
    "Compare los valores de la lluvia (40 a 260) con los de la
    temperatura (21 a 31). "
    "Recuerde lo que se hizo en la Clase 7 antes de entregar
    datos a un modelo. "
    "Guarde también en 'media' y 'desviacion' los valores
```

```
    usados: se necesitan en los retos 6 y 7.",
)

# ============================================================
# 3. Curva de aprendizaje
# ============================================================
fig, (eje1, eje2) = plt.subplots(1, 2, figsize=(12, 4.5))
eje1.plot(historial_perdida, color="red")
eje1.set_title("Pérdida por época")
eje1.set_xlabel("Época")
eje1.grid(True, alpha=0.3)
eje2.plot(historial_exactitud, color="green")
eje2.set_title("Exactitud por época")
eje2.set_xlabel("Época")
eje2.set_ylim(0, 1.05)
eje2.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("curva_aprendizaje.png", dpi=150)
plt.close()
print("Gráfico guardado: curva_aprendizaje.png")

# ============================================================
# >>> RETO 6 <<<  La frontera de decisión
# ============================================================
x1_original = np.linspace(X[:, 0].min(), X[:, 0].max(), 100)
x1_esc = (x1_original - media[0]) / desviacion[0]
# La frontera es la línea donde la neurona duda totalmente
    (probabilidad 0.5).
x2_esc = -(w[0] * x1_esc + b) / w[1]

punto_de_control(
    x2_esc is not None and np.allclose(w[0] * x1_esc + w[1] *
    x2_esc + b, 0, atol=1e-6),
    "RETO 6 (frontera de decisión)",
    "¿Para qué valor de z la sigmoide da exactamente 0.5?
    Escriba esa igualdad con w1, x1, w2, x2 y b, "
    "y despeje x2.",
)
x2_original = x2_esc * desviacion[1] + media[1]

plt.figure(figsize=(9, 6))
plt.scatter(X[y == 1, 0], X[y == 1, 1], color="green",
    edgecolors="black", label="Exitosa (1)")
plt.scatter(X[y == 0, 0], X[y == 0, 1], color="orange",
    marker="s", edgecolors="black", label="No exitosa (0)")
plt.plot(x1_original, x2_original, color="blue", linewidth=2.5,
    label="Frontera de decisión")
plt.ylim(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5)
plt.title("Frontera de decisión – Cultivos de Cartago")
plt.xlabel("Precipitación mensual (mm)")
```

```
plt.ylabel("Temperatura promedio (°C)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("frontera_decision.png", dpi=150)
plt.close()
print("Gráfico guardado: frontera_decision.png")

# ============================================================
# >>> RETO 7 <<<  Usar el modelo con lotes nuevos
# ============================================================
lotes_nuevos = np.array([
    [220.0, 23.5],
    [60.0, 30.0],
    [150.0, 26.0],
])
lotes_nuevos_esc = (lotes_nuevos - media) / desviacion

punto_de_control(
    lotes_nuevos_esc is not None and
    np.allclose(lotes_nuevos_esc, (lotes_nuevos - media) /
    desviacion),
    "RETO 7 (predicción de lotes nuevos)",
    "Los lotes nuevos deben pasar por la MISMA preparación que
    los datos de entrenamiento. "
    "¿Con qué media y qué desviación? ¿Las de los 3 lotes nuevos
    o las de los 120 lotes?",
)

for (lluvia, temperatura), probabilidad in zip(lotes_nuevos,
    predecir_probabilidad(lotes_nuevos_esc, w, b)):
    decision = "EXITOSA" if probabilidad >= 0.5 else "NO
    EXITOSA"
    print(f"Lluvia {lluvia:>5.1f} mm | Temp {temperatura:>4.1f}
    °C -> {probabilidad:.2%} -> {decision}")

print("\n*** ¡Todos los retos superados! Perceptrón completo
    construido desde cero. ***")
```